03. 첫 번째 대화

이 장에서는 Hermes Agent와의 첫 번째 대화를 시작합니다. CLI의 기본 사용법을 익히고, 직접 대화하며 에이전트의 작동 방식을 체험한 뒤, 슬래시 명령어와 음성 모드까지 살펴봅니다.

이 장에서 다루는 내용

- CLI 기본 사용법
- 실습: 첫 대화
- 모델 라우팅
- 슬래시 명령어
- 음성 모드

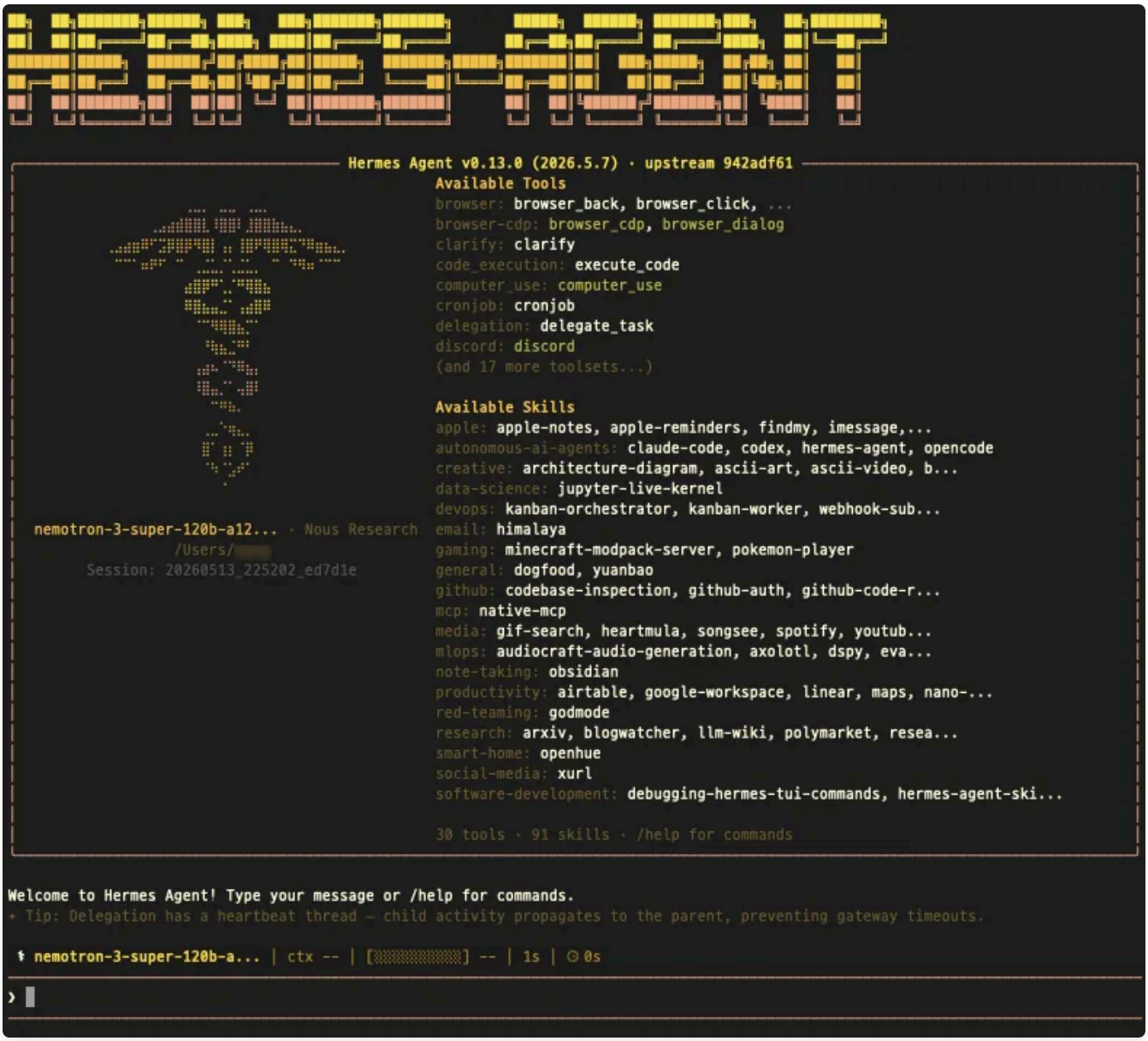
03-1. CLI 기본 사용법

- 첫 실행
- 두 가지 인터페이스: 클래식 REPL과 Ink-based TUI
- 무엇을 시킬 수 있나
  - "내일 회의 준비 좀 도와줘"
  - "이 서버 에러 좀 봐줘"
  - "매일 아침 이것 좀 해줘"
- 알아두면 좋은 것들
  - 종료
  - 일시 중단
  - 실행 옵션
  - 웹 대시보드
  - 이전 대화 이어가기
  - 도구 호출 과정 보기
  - 클릭 가능한 URL (OSC8)
  - @ 컨텍스트 참조

첫 실행

설치가 끝났으면 터미널에서 `hermes` 한 줄이면 됩니다.

```
hermes
```



ASCII 아트 배너와 함께 시작 화면이 뜹니다. 왼쪽에 현재 모델·세션 정보가, 오른쪽에 사용 가능한 도구와 스킬 목록이 보이고, 화면 아래쪽 프롬프트(`>`)에 자연어로 지시하면 됩니다. 화면 맨 아래의 `N tools · M skills` 줄에 환경 기준의 활성 도구·스킬 수가 표시됩니다(위 예시에서는 30개 도구·91개 스킬). 번들 스킬은 약 90개이며, 외부 디렉터리·MCP·스킬 허브로 더 늘릴 수 있습니다.

두 가지 인터페이스: 클래식 REPL과 Ink-based TUI

Hermes에는 진입점이 두 가지 있습니다.

진입점	명령	특징
클래식 REPL	`hermes` (옵션 없이)	가벼움. 스크립트·파이프와 잘 맞고, 비대화형 환경에서도 안전
Ink-based TUI	`hermes --tui`	React/Ink 기반의 풀 화면 TUI. 스티키 컴포저, OSC-52 클립보드, 라이브 스트리밍, LaTeX 렌더링, 라이트/다크 자동 감지 등

TUI는 마우스 휠 라인 스크롤, `/resume` 피커에서 `d` 키로 세션 삭제, `?` 입력 시 미니 헬프 같은 편의 기능이 추가로 들어가 있습니다. 가시 cold start는 lazy import·캐시 등으로 클래식 대비 약 절반 수준입니다. tmux나 SSH 같은 환경에서는 클래식이 더 안정적인 경우도 있으므로, 두 인터페이스를 상황에 맞게 골라 쓰면 됩니다.

무엇을 시킬 수 있나

사람에게 부탁하듯 말하면 됩니다. 에이전트가 어떤 도구를 쓸지, 어떤 순서로 처리할지는 알아서 판단합니다.

"내일 회의 준비 좀 도와줘"

```
> 내일 팀 회의 안건을 정리해줘. 주제는 "신규 프로젝트 킥오프"이고,
> 목표, 일정, 역할 분담, 리스크, 다음 단계 정도로 항목을 잡아줘.
> 파일로 저장도 해줘.
```

에이전트가 안건을 작성하고, 파일로 저장까지 해줍니다. 사용자는 **원하는 결과물만 말하면** 됩니다. 같은 종류의 작업을 반복하면, Hermes Agent는 그 절차를 스킬로 저장해서 다음에는 더 빠르게 처리합니다. (→ 7장)

"이 서버 에러 좀 봐줘"

```
> 서버 에러 로그에서 최근에 뭐가 문제인지 파악하고,
> 가장 빈번한 문제 3가지와 해결 방안을 알려줘
```

에이전트가 로그 파일을 찾아 읽고, 에러 패턴을 분류하고, 해결 방안까지 제시합니다. 구체적인 파일 경로나 분석 명령을 몰라도, 원하는 것만 말하면 에이전트가 방법을 찾습니다.

"매일 아침 이것 좀 해줘"

```
> 매일 아침 9시에 GitHub 이슈 중 버그를 요약해서 Telegram으로 보내줘
```

이 한마디로 자동화가 설정됩니다. "매일 아침 9시"라는 일정 해석, 이슈 필터링, 메시지 전달 — 모든 세부사항은 에이전트가 처리합니다. 결과는 Telegram, Discord, Slack 등으로 받을 수 있습니다. (→ 10장)

이것이 단순한 챗봇이나 명령줄 도구와 다른 점입니다. **사용자는 원하는 것만 말하고, 에이전트가 방법을 찾아 실행합니다.** 게다가 사용할수록 사용자의 패턴을 학습하여 점점 빨라집니다.

> OpenClaw에서 전환하려면
>
> `hermes claw migrate` 명령으로 OpenClaw의 설정, 메모리, API 키를 그대로 가져올 수 있습니다. 기존 환경을 버리지 않고 전환할 수 있습니다.

알아두면 좋은 것들

종료

대화를 끝내려면 `/quit` 또는 `/q`를 입력합니다. Ctrl+D도 작동합니다.

일시 중단

Ctrl+Z를 누르면 Hermes 프로세스가 일시 중단됩니다. 셸로 돌아가 다른 작업을 한 뒤, `fg` 명령으로 다시 돌아올 수 있습니다. Unix 계열(macOS, Linux)에서만 지원되며 Windows에서는 작동하지 않습니다.

실행 옵션

대화형 세션 대신 한 번만 질문하고 끝낼 수도 있습니다.

```
hermes chat -q "오늘 날짜가 며칠이야?"
```

스크립트나 파이프에 끼워 넣을 때는 더 좋은 방법이 있습니다. `hermes -z` (`--oneshot`) 모드는 배너·스피너·도구 미리보기·세션 ID 줄을 모두 끄고, **최종 응답 텍스트만** stdout으로 내보냅니다. 승인 프롬프트도 자동으로 우회되므로 비대화형 자동화에 적합합니다.

```
hermes -z "오늘 사람들이 SNS에서 가장 많이 얘기한 IT 키워드 3개를 한 줄씩 나열해줘"
```

`-z`에 `-m/--model`, `--provider`, `-t/--toolsets` 같은 옵션을 함께 쓰면 인보케이션 단위로 모델과 도구 셋을 바꿀 수 있습니다(`HERMES_INFERENCE_MODEL`·`HERMES_INFERENCE_PROVIDER` 환경 변수도 동일하게 작동).

특정 모델로 실행하려면 `--model` 옵션을 사용합니다. 해당 모델이 Ollama에 미리 다운로드되어 있어야 합니다(`ollama pull <모델명>`).

```
hermes chat --model "qwen3:14b"
```

턴 수를 제한하려면 `--max-turns` 옵션을 사용합니다. 자동화 스크립트에서 에이전트가 무한 루프에 빠지는 것을 방지할 때 유용합니다.

```
hermes chat --max-turns 10
```

> 💡 ***/fast 모드**: 대화 중 `/fast` 를 입력하면 OpenAI Priority Processing 또는 Anthropic Fast 티어를 통해 응답 지연을 줄입니다. 같은 모델·같은 요청이지만 우선 처리 큐로 보내는 방식이며, OpenAI(GPT·Codex)와 Anthropic(Claude) 모델에만 적용됩니다. 자세한 내용은 3-3절을 참고하세요.*

웹 대시보드

CLI를 켜는 대신 브라우저로 설정·세션·플러그인을 관리할 수도 있습니다.

```
hermes dashboard           # 기본 포트 9119, 자동으로 브라우저 오픈
hermes dashboard --status  # 실행 중인 대시보드 목록
hermes dashboard --stop    # 모든 대시보드 종료
```

대시보드는 기본적으로 `127.0.0.1:9119` 에 바인드되어 로컬 전용입니다. Models 탭에서 메인·보조 모델을 그대로 바꿀 수 있고, Plugins 탭에서 플러그인 활성화/비활성화·인증·설치/삭제를 한자리에서 합니다. `--insecure` 플래그로 비루프백 호스트에 노출할 수도 있지만 API 키가 LAN에 노출되므로 신뢰된 네트워크에서만 사용하세요.

이전 대화 이어가기

대화 세션은 자동 저장됩니다. 터미널을 닫았다가 다시 열어도 이어서 대화할 수 있습니다.

```
hermes -c              # 마지막 세션 이어서
```

도구 호출 과정 보기

에이전트가 어떤 도구를 왜 쓰는지 보고 싶다면, 대화 중에 `/verbose` 를 입력하세요. 도구 호출 과정이 상세히 표시됩니다. 처음에는 켜두고 관찰하는 것을 권장합니다.

클릭 가능한 URL (OSC8)

v0.14부터 에이전트 출력의 URL은 단순 텍스트가 아니라 **OSC8 하이퍼링크**로 렌더링됩니다. iTerm2·Kitty·Ghostty·최신 Windows Terminal 같은 OSC8 지원 터미널에서 호버 강조가 뜨고, 클릭하면 브라우저로 열립니다. 긴 URL을 복사·붙여넣기 하다가 일부가 잘려 나가는 사고가 사라집니다. OSC8을 지원하지 않는 터미널에서는 평범한 텍스트로 보입니다.

@ 컨텍스트 참조

메시지에 `@` 접두사를 붙이면 파일, 코드, 변경 사항 등을 대화에 직접 주입할 수 있습니다.

```
> @file:src/main.py 이 코드에서 버그를 찾아줘
> @diff 이 변경 사항을 리뷰해줘
> @git:5 최근 5개 커밋을 요약해줘
```

지원하는 참조 형식은 다음과 같습니다.

참조	설명
`@file:경로`	파일 내용 주입
`@file:경로:10-50`	특정 줄 범위만 주입
`@folder:경로`	디렉터리 구조와 파일 주입
`@diff`	커밋되지 않은 변경 사항
`@staged`	스테이징된 변경 사항
`@git:N`	최근 N개 커밋
`@url:주소`	웹 페이지 내용

토큰 예산 보호도 내장되어 있어서, 참조한 내용이 컨텍스트의 25%를 넘으면 경고가, 50%를 넘으면 차단이 됩니다. 파일을 복사-붙여넣기하는 대신 `@file:`로 가리키면 훨씬 효율적입니다.

이 밖의 슬래시 명령어와 단축키는 3-3절에서 자세히 다룹니다.

웹 UI도 가능합니다

터미널이 불편하다면, Hermes Agent에 내장된 OpenAI 호환 API 서버를 통해 Open WebUI 같은 웹 프런트엔드를 연결할 수 있습니다. 다만 이 기능은 아직 초기 단계이며, CLI의 `/verbose`처럼 사고 과정을 관찰하기는 어렵습니다.

03-2. 실습: 첫 대화

- 실습 1: 말 걸어 보기
- 실습 2: 정보 조사
- 실습 3: 작업 맡기기
- 관찰 포인트

이 절에서는 Hermes Agent와 직접 대화하며 기본적인 작동 방식을 체험합니다. 사람에게 부탁하듯 자연스럽게 말하면 됩니다.

실습 환경

이 실습은 「클라우드 API 연동」 절에서 골라 둔 **OpenRouter의 무료 모델** `nvidia/nemotron-3-super-120b-a12b:free` 를 사용합니다. 새로 가입했고 크레딧이 0인 상태에서도 그대로 따라 할 수 있도록 한 선택입니다. 다른 모델을 사용 중이라면 `/model` 로 전환할 수 있고, 실습 결과는 모델에 따라 달라질 수 있습니다. 한국어 품질이 더 좋은 옵션은 부록 「로컬 LLM 가이드」를 참고하세요.

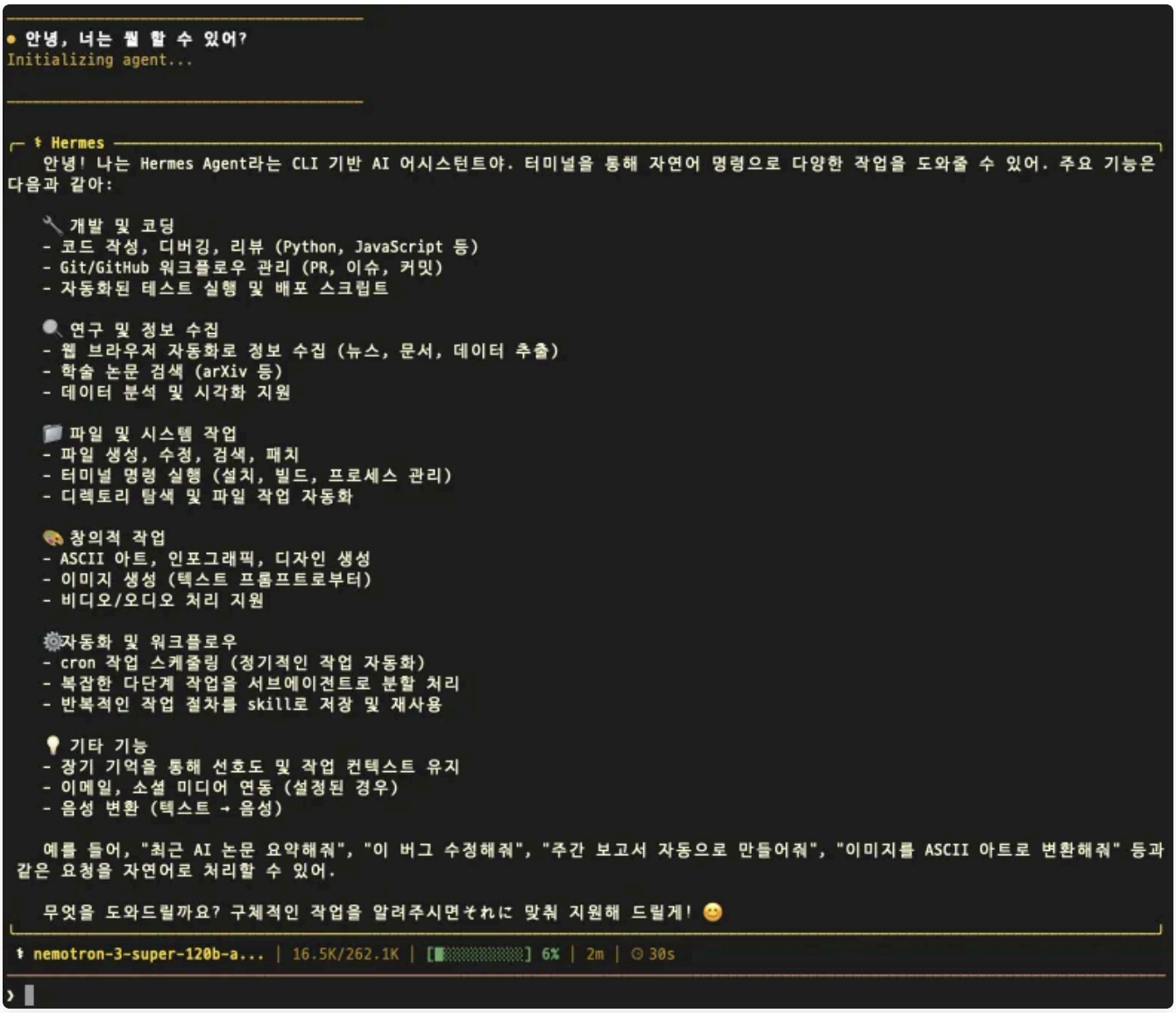
> ⚠️ ***무료 모델은 입출력이 모델 학습에 사용됩니다***: *OpenRouter* `:free` *엔드포인트는 "모든 프롬프트와 응답이 프로바이더의 모델·제품·서비스 개선 목적으로 로깅된다"고 공식 고지합니다. 시험·학습 용도로만 사용하고,* ***개인 정보·기밀 정보·업무 데이터는 입력하지 마세요****. 실제 업무에는 유료 모델이나 로컬 LLM(부록 「로컬 LLM 가이드」)을 사용합니다.*

실습 1: 말 걸어 보기

Hermes Agent를 시작하고, 편하게 말을 걸어 봅니다.

```
$ hermes

> 안녕, 너는 뭘 할 수 있어?
```



에이전트가 잠시 생각한 뒤, 자신이 할 수 있는 일을 자연스럽게 설명합니다. 파일 작업·터미널·웹 검색 같은 기본 도구와 설치 직후 활성화된 번들 스킬이 함께 소개됩니다. 사용자가 MCP 서버를 등록하거나 새 스킬을 추가하면 이 목록은 그만큼 풍성해지고, 같은 질문에도 답의 모양이 달라집니다(7장 스킬·8장 MCP 참고). 단순한 챗봇이 아니라 **실제로 행동하는** 에이전트라는 것을 첫 대화에서 바로 느낄 수 있습니다.

실습 2: 정보 조사

에이전트에게 무언가를 조사해 달라고 부탁해 봅니다.

```
> NousResearch가 어떤 회사인지 간단히 알려줘
```

에이전트가 웹 검색 도구를 사용하여 정보를 찾고, 요약해 줍니다. 직접 검색해서 여러 페이지를 읽는 대신, 에이전트에게 맡기면 한 번에 정리된 답을 받을 수 있습니다.

웹 검색이 안 된다면

모델에 따라 웹 검색 결과를 도구 호출로 깔끔히 처리하지 못하는 경우가 있습니다. 무료 모델이나 14B 이하의 로컬 LLM에서 자주 나타납니다. 이때는 "검색하지 말고 네가 아는 대로 알려줘"라고 하면 학습 데이터 기반으로 답합니다. 웹 검색 기능은 모델의 도구 호출 능력에 크게 의존하며, 14B 이상 모델이나 프론티어 클라우드 모델에서 더 안정적입니다.

실습 3: 작업 맡기기

이번에는 단순한 질문이 아니라, 결과물을 만들어 달라고 부탁합니다. 이 실습부터는 `/verbose`를 켜두면 에이전트가 어떤 도구를 골라 쓰는지 직접 볼 수 있습니다.

```
> /verbose
> 러닝 초보자를 위한 4주 훈련 계획을 짜서 파일로 저장해줘.
```

`/verbose`는 한 번 입력할 때마다 표시 수준이 순환합니다(off → new → all → verbose). 처음 누르면 `new` 모드(새로 호출된 도구 표시)가 되고, 끄려면 `off`가 될 때까지 세 번 더 입력합니다.

에이전트가 스스로 판단해서 필요한 도구를 고르고, 계획을 세우고, 파일로 저장합니다. 웹 검색이 필요하다고 판단하면 알아서 검색하고, 아니면 바로 작성에 들어갑니다. `/verbose`를 켜두었으므로 이 과정이 화면에 표시됩니다. 일반 챗봇은 텍스트를 화면에 보여주기만 하지만, Hermes Agent는 **도구를 조합해서 실제로 파일을 생성**합니다.

이어서 수정을 요청해 봅니다.

```
> 주 3회로 줄이고 스트레칭 루틴도 넣어줘
```

에이전트가 직전에 만든 파일을 기억하고 있어서, 어떤 파일이라고 말하지 않아도 그 파일을 고칩니다.

같은 질문을 다른 모델로 던져 보고 싶다면 `/model`로 전환할 수 있습니다. 모델 라우팅과 피커 사용법은 다음 절(3-3 모델 라우팅)에서 자세히 다룹니다.

관찰 포인트

첫 대화에서 주목할 점들:

- **자연어 이해**: 딱딱한 명령어가 아니라 일상적인 말로 지시해도 의도를 파악합니다.
- **도구 선택**: 실습 3에서 `/verbose`로 관찰한 것처럼, 에이전트가 어떤 도구를 어떤 순서로 쓸지 스스로 판단합니다. 검색이 필요하면 검색하고, 파일을 써야 하면 파일을 씁니다.
- **컨텍스트 유지**: "주 3회로 줄여줘"처럼, 앞선 대화를 기억하고 있는지 확인하세요.

- **챗봇과의 차이**: 단순히 텍스트를 생성하는 것이 아니라, 도구를 조합해서 파일을 저장하고 명령을 실행하는 등 **실제로 행동한다**는 점을 체감하세요.

03-3. 모델 라우팅

- 모델·프로바이더가 결정되는 흐름
- 라이브 모델 전환 (/model)
  - hermes model과 /model의 차이
- Fast Mode (/fast)
- 비용 최적화 전략
- 비전 지원 수동 선언 (supports_vision)
- 보조 모델 슬롯의 기본 라우팅
- OpenRouter 라우팅 옵션
  - 제공자 라우팅 — 같은 모델, 다른 인프라
  - 모델 라우팅 — OpenRouter가 모델 자체를 선택

3-2절에서는 한 가지 모델로 실습했습니다. 이 절에서는 모델을 고르고 전환하는 다양한 방법, 그리고 비용·속도를 최적화하는 전략을 살펴봅니다.

> OpenRouter 라우팅과의 구분
>
> OpenRouter도 "라우팅"이라는 단어를 쓰지만 이 절의 주제와 다릅니다. OpenRouter에는 같은 모델을 여러 인프라 제공자 중 어디서 실행할지 고르는 **제공자 라우팅**과, 모델 자체를 자동으로 선택하는 **모델 라우팅**(auto, pareto-code) 두 계층이 있습니다. 이 절에서 주로 다루는 것은 **Hermes 클라이언트** 쪽에서 사용자가 프로바이더·모델을 선택하고 전환하는 방법이며, OpenRouter 라우팅의 설정 방법은 이 절 하단의 "OpenRouter 라우팅 옵션"에서 다룹니다.

모델·프로바이더가 결정되는 흐름

요청이 어느 모델로 가는지는 다음 우선순위로 결정됩니다(높은 것이 우선).

1. **CLI 플래그**: `--provider`, `--model` (또는 `-m`)
2. **config.yaml**: `model.provider`, `model.default`
3. **환경 변수**: `HERMES_INFERENCE_PROVIDER`, `HERMES_INFERENCE_MODEL`
4. **auto 탐지**: 위 셋 모두 없으면 사용 가능한 자격 증명을 자동으로 찾습니다

한 번만 다른 모델로 실행하려면 CLI 플래그가 편합니다.

```
hermes chat --provider anthropic -m claude-sonnet-4-6
hermes -z --provider custom -m qwen3:14b "이 코드를 리팩터링해줘"
```

영구적으로 바꾸려면 `hermes model`로 인터랙티브 피커를 띄우거나, `config.yaml`의 `model.provider`·`model.default`를 직접 수정합니다(2-3절 참고).

라이브 모델 전환 (`/model`)

세션 도중에 모델·프로바이더를 즉석에서 갈아 끼울 수 있습니다.

```
> /model
```

인자 없이 부르면 인터랙티브 피커가 뜨고, 인증된 프로바이더가 모두 나옵니다. 한 번에 모델명을 같이 줘도 됩니다.

```
> /model anthropic/claude-sonnet-4-6
> /model gpt-5.5 --provider codex
> /model qwen3:14b --provider custom --global    # config.yaml 영구 반영
```

이 명령은 CLI뿐 아니라 텔레그램·디스코드·슬랙 같은 게이트웨이 플랫폼에서도 그대로 작동합니다. 텔레그램·디스코드에서는 인라인 버튼이 달린 모델 피커가 뜹니다. 애그리게이터(OpenRouter·Nous Portal 등)를 쓰는 경우 가능한 한 같은 애그리게이터 안에서 해석해 주고, 안 되면 다른 프로바이더로 자동 폴백합니다.

`hermes model`과 `/model`의 차이

이름은 비슷하지만 두 명령은 쓰임이 다릅니다. 정리하면 다음과 같습니다.

비교 항목	`hermes model`	`/model`
실행 시점	Hermes 진입 전, 셸에서 실행	Hermes 세션 안에서 슬래시 명령으로 실행
기본 화면	프로바이더 → 자격 증명 입력 → 모델 선택 순서의 셋업 마법사	인증된 프로바이더만 보여 주는 인터랙티브 피커
자격 증명	미인증 프로바이더를 고르면 그 자리에서 API 키 프롬프트	키 입력 단계 없음. 이미 인증된 프로바이더 중에서만 선택
적용 범위	`config.yaml` 에 저장. 다음 실행 세션부터 반영	현재 세션의 에이전트에 즉시 반영. `--global` 을 붙이면 `config.yaml` 까지 함께 반영
자주 쓰는 상황	처음 설정하거나 새 프로바이더를 추가할 때	대화 도중 비용·속도 균형을 위해 모델을 갈아 끼울 때

요컨대 영구 설정이 목적이면 `hermes model` 또는 `/model <이름> --global` 이 같은 결과를 만들고, `/model <이름>` 단독은 "이 세션에서만"이라는 점이 가장 큰 차이입니다.

Fast Mode (`/fast`)

OpenAI Priority Processing과 Anthropic의 Fast 티어를 활용해 응답 지연을 줄이는 토글입니다. 같은 모델로 같은 요청을 보내되, **우선 처리 큐**로 보냅니다.

```
> /fast            # 토글
> /fast status     # 현재 상태
> /fast on         # 켜기
> /fast off        # 끄기
```

OpenAI(GPT 시리즈·Codex)와 Anthropic(Claude 시리즈) 모델 전반에 적용됩니다. 토큰 단가가 약간 더 비쌀 수 있는 대신 체감 응답 속도가 크게 개선되므로, 코드 리뷰처럼 인터랙션이 빠르게 오가는 상황에 유용합니다.

비용 최적화 전략

모델 비용을 줄이면서 대화 품질을 유지하는 방법은 몇 가지가 있습니다.

1. 보조 모델 활용

이미지 분석·웹 요약·컨텍스트 압축 같은 부수 작업에 경량 모델을 지정합니다. 주 모델은 클라우드 프론티어 모델을 유지하면서, 보조 슬롯(`auxiliary`)에 Gemini Flash 같은 저렴한 모델을 배정하면 비용이 크게 줄어듭니다(2-5절 참고).

2. 서브에이전트 모델 분리

`delegation.model`에 경량 모델을 지정하면, 주 모델(예: Claude Sonnet)이 복잡한 판단을 하고 서브에이전트(예: Llama 8B)가 단순 작업을 처리합니다. 주 모델이 작업 지시를 생성하고, 서브에이전트가 파일 생성·검색 같은 반복 작업을 수행하는 패턴입니다.

3. 세션 단위 모델 전환

간단한 질의에는 `/model`로 경량 모델로 전환했다가, 복잡한 작업이 필요할 때 다시 프론티어 모델로 돌아오는 수동 전환도 효과적입니다. `--global` 플래그를 빼면 현재 세션에서만 적용됩니다.

4. OpenRouter Pareto Code 라우터

OpenRouter 프로바이더를 사용한다면, 모델을 `openrouter/pareto-code`로 지정해 비용 최적화를 OpenRouter에 위임할 수 있습니다. 코딩 벤치마크 점수의 하한선(`min_coding_score`, 0~1)을 정해 두면, 그 기준을 충족하는 모델 중 가장 저렴한 모델을 OpenRouter가 자동으로 골라 줍니다.

```
# config.yaml
model:
  default: "openrouter/pareto-code"

openrouter:
  min_coding_score: 0.65      # 이 점수 이상인 코딩 모델 중 최저가 선택
```

점수를 높이면 품질이 보장되는 대신 비싼 모델이 선택되고, 낮추면 저렴한 모델이 선택됩니다. 생략하면 최상위 티어가 기본값입니다. 고정 모델 지정과 달리 새 모델이 OpenRouter에 추가될 때마다 자동으로 후보에 포함되므로, 가격 경쟁이 반영됩니다.

프론티어 모델로 판단하고 코딩 작업만 pareto-code로 보내는 구성도 가능합니다. `delegation.model`에 pareto-code를 지정하면 `delegate_task`로 위임된 서브에이전트가 pareto-code를 사용합니다. `openrouter.min_coding_score`는 서브에이전트에 자동 상속됩니다.

```yaml
model:
  default: "anthropic/claude-opus-4.8"
  provider: "openrouter"

delegation:
  model: "openrouter/pareto-code"
  provider: "openrouter"

openrouter:
  min_coding_score: 0.65
```

이 구성에서는 Opus가 분석·계획을 담당하고, 파일 생성·코드 수정 같은 반복 작업을 서브에이전트에 위임할 때 pareto-code가 비용 효율적인 모델을 골라 줍니다.

5. `/fast` 모드

비용은 약간 더 들지만, 같은 모델의 우선 처리 큐를 사용해 대기 시간을 줄입니다. 인터랙션이 빈번한 작업에서 체감 효과가 큽니다.

삭제된 smart_model_routing과 대체 경로

이전 버전에는 메시지 복잡도에 따라 cheap_model과 기본 모델을 자동 분배하는 `smart_model_routing` 설정이 있었습니다. opt-in(기본 비활성)이었고 실사용이 적어, v0.13.0에서 제거되었습니다(`424e9f36b`). 판정 로직이 영어 키워드 44개에만 의존하는 규칙 기반이었으므로, 한국어 입력에서는 사실상 분기가 작동하지 않는 한계도 있었습니다.

대체 경로로 `openrouter/pareto-code` 가 도입되었습니다(`c7f0aab94`). 클라이언트 측 규칙 대신 OpenRouter 서버 측 벤치마크 기반으로 비용을 최적화하므로 언어와 무관하게 작동합니다.

비전 지원 수동 선언 (`supports_vision`)

models.dev에 등록되지 않은 커스텀·로컬 모델은 기본적으로 비전 미지원으로 분류되어, 이미지를 보내도 API 호출 전에 이미지 콘텐츠가 자동 제거됩니다. LLaVA, Qwen-VL 같은 로컬 비전 모델을 사용할 때 이 기본 분류를 재정의할 수 있습니다.

```
# config.yaml — 현재 기본 모델에 적용
model:
  supports_vision: true

# 특정 프로바이더의 특정 모델에만 적용
providers:
  my-vllm:
    models:
      my-llava:
        supports_vision: true
```

이 설정은 models.dev 조회를 건너뛰고 이미지 전처리 단계에서 바로 비전 가능으로 판정합니다.

보조 모델 슬롯의 기본 라우팅

이미지 분석·웹 페이지 요약·컨텍스트 압축·세션 검색 같은 부수 작업에 쓰이는 보조 모델 슬롯은 별도로 고르지 않으면 (`provider: "auto"`) **메인 프로바이더를 따라갑니다**. 메인이 OpenRouter 같은 aggregator라면 메인 모델 그대로 호출되고, Anthropic Direct·z.ai·Kimi 같은 직접 키 프로바이더이면 같은 프로바이더의 가벼운 보조 모델(예: Anthropic은 Haiku, z.ai는 GLM Flash)로 자동 매핑됩니다. 가벼운 보조 작업이 메인 모델 그대로 호출돼 비용이 의외로 큰 상황이면 `hermes model`의 보조 모델 설정 화면(또는 웹 대시보드의 Models 탭)에서 슬롯별로 가벼운 모델을 직접 지정합니다. 자세한 슬롯 목록과 의미는 2-5절을 참고하세요.

OpenRouter 라우팅 옵션

OpenRouter를 프로바이더로 사용할 때는 **모델 라우팅**과 **제공자 라우팅**, 두 가지 계층이 있습니다.

제공자 라우팅 — 같은 모델, 다른 인프라

같은 모델(예: `claude-sonnet-4`)을 Anthropic 직접, AWS Bedrock, Google Vertex 등 여러 인프라 제공자가 호스팅합니다. OpenRouter는 기본적으로 가용성과 부하를 보고 자동으로 고르지만, `config.yaml`의 `provider_routing` 블록으로 제어할 수 있습니다.

```
provider_routing:
  sort: "throughput"          # price | throughput | latency
  data_collection: "deny"     # 데이터 보존 제공자 제외
```

주요 옵션과 설정 방법은 2-5절의 "OpenRouter 프로바이더 라우팅" 항목을 참고하세요.

모델 라우팅 — OpenRouter가 모델 자체를 선택

OpenRouter에는 모델 자체를 자동으로 고르는 메타 모델(라우터)도 있습니다.

모델 ID	방식	용도
`openrouter/pareto-code`	코딩 점수 하한선 + 최저가	코딩 에이전트 비용 최적화
`openrouter/auto`	NotDiamond 기반 프롬프트 분석	범용 자동 선택

`openrouter/pareto-code`는 Hermes 소스에 `plugins` 전달 로직이 구현돼 있어, `config.yaml`의 `openrouter.min_coding_score`와 연동됩니다(위 "비용 최적화 전략" 4번 참고).

`openrouter/auto`는 OpenRouter의 공식 기능으로, 프롬프트를 분석해 ~38개 후보 모델 중 최적 모델을 매 요청마다 동적으로 선택합니다. `/model openrouter/auto`로 전환할 수 있지만, 코딩 에이전트에서는 매 턴 모델이 바뀌면 tool calling 포맷·reasoning 스타일이 달라져 안정성이 떨어질 수 있습니다. 코딩 작업에는 모델을 고정하거나 `pareto-code`를 사용하는 편이 낫습니다.

<fieldset>
<legend>라우팅 계층 정리</legend>

정리하면 세 가지 계층이 있습니다.

- **Hermes 모델 선택**: 사용자가 `/model`이나 `config.yaml`로 직접 고르는 것
- **OpenRouter 모델 라우팅**: `openrouter/auto`·`openrouter/pareto-code`처럼 OpenRouter가 모델을 골라 주는 것
- **OpenRouter 제공자 라우팅**: 이미 정해진 모델을 어떤 인프라에서 실행할지 고르는 것

일반적으로는 첫 번째(모델 고정) + 세 번째(제공자 라우팅으로 속도·비용 조절)의 조합이 코딩 에이전트에서 가장 안정적입니다.
</fieldset>

03-4. 슬래시 명령어

- 세션 관리
- 백그라운드·다중 작업
- 모델·응답 작동
- 표시·입력 작동
- 도구·스킬·MCP
- 정보·도움말
- 게이트웨이 전용
- 종료
- 퀵 커맨드

Hermes Agent는 대화 중 `/`로 시작하는 슬래시 명령어를 제공합니다. CLI와 메시징 플랫폼 모두에서 사용할 수 있으며, 접두사 매칭을 지원합니다(예: `/ver` 만 입력해도 `/verbose`로 확장).

명령어는 `~/hermes-agent/hermes_cli/commands.py`의 `COMMAND_REGISTRY`가 단일 진실의 원천입니다. 이 절은 v0.13.0 기준으로 정리한 것입니다.

세션 관리

명령어	설명
`/new` (별칭: `/reset`)	새 세션 시작 (세션 ID와 히스토리 초기화)
`/clear`	화면 지우고 새 세션 시작
`/redraw`	터미널 화면이 깨졌을 때 강제 재페인트
`/history`	대화 히스토리 표시
`/save`	현재 세션 저장
`/retry`	마지막 메시지 재전송
`/undo`	마지막 사용자/어시스턴트 교환 제거
`/title [name]`	세션 제목 설정/표시
`/branch [name]` (별칭: `/fork`)	현재 세션을 분기해 다른 경로 탐색
`/compress [focus]`	컨텍스트 수동 압축. 주제를 인자로 주면 그 주제 중심으로 요약
`/compress here [N]`	경계 인식형 부분 압축. 최근 N개 교환은 그대로 두고 그 이전만 요약 (5장 참고)
`/rollback [number]`	파일 시스템 체크포인트 목록/복원
`/snapshot [create|restore <id>|prune]` (별칭: `/snap`)	Hermes 설정·상태의 스냅숏
`/stop`	실행 중인 백그라운드 프로세스 모두 종료
`/resume [name]`	이전 세션 재개
`/status`	세션 정보 표시
`/profile`	활성 프로파일과 홈 디렉터리 표시(전환은 CLI에서, 11-6절 참고)
`/sessions`	이전 세션 목록 탐색 및 재개

명령어	설명
`/handoff <target>`	현재 세션을 라이브 이관 — 메시지·도구 호출·컨텍스트를 전부 보존한 채 다른 모델, 성격, 프로파일, 또는 메시징 플랫폼(Telegram, Discord 등)으로 옮김. 디버깅 중 빠른 모델에서 깊은 추론 모델로 넘기는 식으로 사용
`/sethome` (별칭: `/set-home`)	현재 채널을 홈 채널로 지정(게이트웨이 전용)

백그라운드·다중 작업

여러 작업을 동시에 진행하거나, 실행 중인 에이전트를 살짝 돌려세우는 명령어들입니다.

명령어	설명
`/background <prompt>` (별칭: `/bg`, `/btw`)	서브에이전트로 백그라운드 작업 시작. 메인 대화는 계속 진행
`/queue <prompt>` (별칭: `/q`)	다음 턴에 처리할 프롬프트를 큐에 넣음(현재 턴은 끊지 않음)
`/steer <prompt>`	실행 중인 에이전트가 다음 도구 호출 후에 받아 보도록 메시지 주입(인터럽트 없음, 프롬프트 캐시 유지)
`/goal [text|pause|resume|clear|status]`	영속 목표 설정 — 매 턴 후 백그라운드에서 진행 점검(Ralph loop). 한 턴에 끝나지 않는 작업을 며칠에 걸쳐 누적 진행할 때 사용
`/agents` (별칭: `/tasks`)	활성 서브에이전트와 진행 중 작업 표시

모델·응답 작동

명령어	설명
`/model [model] [--provider name] [--global]` (별칭: `/provider`)	모델·프로바이더 전환. 인자 없이 부르면 인터랙티브 피커가 뜨고, 모든 게이트웨이 플랫폼(텔레그램·디스코드·슬랙)에서도 작동
`/fast [normal|fast|status|on|off]`	Fast Mode 토글 — OpenAI Priority Processing / Anthropic Fast Mode 사용 여부
`/codex-runtime [on|off|auto]`	OpenAI/Codex 모델 턴을 Codex app-server로 위임하는 런타임 전환(4-4절)
`/reasoning [level|show|hide]`	추론 노력 수준(none/minimal/low/medium/high/xhigh) 또는 추론 과정 표시/숨김
`/footer [on|off|status]`	게이트웨이 최종 응답에 런타임 메타데이터 푸터(모델·토큰·소요시간) 표시 토글
`/yolo`	위험 명령 자동 승인 토글(세션 한정, 프로세스 종료 시 초기화)

표시·입력 작동

명령어	설명
`/verbose`	도구 표시 수준 순환 (off → new → all → verbose)
`/voice [on|off|tts|status]`	음성 모드 토글(자세한 내용은 3-5절)
`/busy [queue|steer|interrupt|status]`	작업 중 Enter 키 작동 변경
`/indicator [kaomoji|emoji|unicode|ascii]`	TUI의 busy 인디케이터 스타일
`/skin [name]`	디스플레이 테마(스킨) 변경
`/statusbar` (별칭: `/sb`)	컨텍스트/모델 상태바 토글
`/personality [name]`	성격 프리셋 적용(없이 부르면 현재 표시)
`/usage`	현재 세션 토큰 사용량과 레이트 리밋
`/insights [days]`	사용 통계 분석

성격 프리셋(14가지): `helpful`, `concise`, `technical`, `creative`, `teacher`, `kawaii`, `catgirl`, `pirate`, `shakespeare`, `surfer`, `noir`, `uwu`, `philosopher`, `hype`. `/personality none` 또는 `/personality default`로 초기화합니다.

> /personality는 세션을 넘어 지속됩니다
>
> 공식 문서에는 세션 수준의 임시 변경이라고 되어 있지만, 소스 코드에서는 config 파일에 저장되어 다음 세션에도 유지됩니다.

도구·스킬·MCP

명령어	설명
`/tools [list|disable|enable] [name...]`	도구 활성/비활성
`/toolsets`	사용 가능한 도구 세트 목록
`/skills [search|browse|inspect|install]`	스킬 검색·설치·관리
`/<skill-name>`	특정 스킬 실행
`/cron [list|add|create|edit|pause|resume|run|remove]`	크론 작업 관리
`/curator [status|run|pause|resume|pin|unpin|restore]`	백그라운드 스킬 큐레이터(7장 참고)
`/kanban [list|show|create|claim|complete|...]`	다중 프로필 협업 보드
`/reload-mcp` (별칭: `/reload_mcp`)	MCP 서버 재로드. 프롬프트 캐시 무효화 비용 경고 후 진행
`/reload-skills` (별칭: `/reload_skills`)	`~/.hermes/skills/` 재스캔 — 새로 설치하거나 삭제한 스킬 즉시 반영
`/reload`	`.env` 변수 재로드(현재 세션에 반영)
`/browser [connect|disconnect|status]`	Chrome CDP 연결 관리
`/plugins`	설치된 플러그인 상태
`/platforms` (별칭: `/gateway`)	게이트웨이/메시징 플랫폼 상태

정보·도움말

명령어	설명
`/help`	사용 가능한 명령어 도움말
`/whoami`	현재 사용자의 슬래시 명령 접근 수준(admin/user) 표시
`/commands [page]`	전체 명령어/스킬 목록 페이지네이션(게이트웨이 전용)
`/config`	현재 설정 표시
`/copy [number]`	마지막 어시스턴트 응답을 클립보드로
`/paste`	클립보드 이미지를 다음 프롬프트에 첨부
`/image <path>`	로컬 이미지 파일을 다음 프롬프트에 첨부
`/gquota`	Google Gemini Code Assist 할당량
`/debug`	시스템 정보 + 로그를 묶어 공유용 디버그 리포트 업로드

게이트웨이 전용

메시징 플랫폼에서 추가로 쓰는 명령어:

명령어	설명
`/topic [off|help|session-id]`	Telegram DM 토픽 세션 관리
`/approve [session|always]`	보류 중인 위험 명령 승인
`/deny`	보류 중인 위험 명령 거부
`/restart`	진행 중 작업 정리 후 게이트웨이 재시작
`/update`	Hermes Agent 업데이트

종료

명령어	설명
`/quit` (별칭: `/exit`)	CLI 종료(`/q` 는 `/queue` 별칭이므로 종료에는 `/quit` 사용)

> 💡 ***사라진 명령어**: 이전 버전에 있던 `/plan` 은 v0.11에서 제거됐습니다. `/provider` 는 별도 명령이 아니라 `/model` 의 별칭으로 통합됐습니다(인터랙티브 모델 피커가 뜹니다).*

퀵 커맨드

`config.yaml` 에 사용자 정의 명령어를 등록하면 LLM을 거치지 않고 바로 셸 명령을 실행할 수 있습니다.

```
quick_commands:
  status:
    type: exec
    command: systemctl status hermes-agent
  disk:
    type: exec
    command: df -h /
```

등록 후 `/status` , `/disk` 로 실행합니다. `type` 은 `exec` (셸 명령 실행)과 `alias` (다른 명령 별칭) 두 가지를 지원합니다. 토큰 소비 없이 즉시 결과를 볼 수 있습니다.

03-5. 음성 모드

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스 코드(`tools/voice_mode.py` , `tools/tts_tool.py` , `tools/transcription_tools.py` , `hermes_cli/voice.py` , `hermes_cli/config.py`)를 기준으로 작성했습니다. STT/TTS 프로바이더·기본 모델·설정 기본값·슬래시 명령은 소스에서 확인했으나, 실제 마이크 입력·음성 재생 · `/voice status` 출력은 실행 테스트로 검증하지 않았습니다.*

- 추가 설치
- 빠르게 시작하기
- STT: 음성을 텍스트로
  - 로컬 모델 크기 선택
- TTS: 텍스트를 음성으로
- TUI 음성 모드
- 설정
  - 무음 감지 조정
  - 녹음 키 변경
- 실습: 음성으로 대화하기
- 음성 모드 슬래시 명령어
- 메신저에서의 음성

Hermes Agent는 메신저를 거치지 않고 마이크와 스피커만으로 직접 대화할 수 있습니다. 키보드를 치는 대신 말로 지시하고, 에이전트가 음성으로 답합니다. 코딩하면서 손을 떼기 싫을 때, 산책하면서 아이디어를 정리할 때, 또는 단순히 타이핑이 번거로울 때 유용합니다.

추가 설치

음성 모드를 사용하려면 오디오 관련 패키지가 필요합니다.

```
# Python 패키지
uv pip install -e ".[voice]"

# macOS 시스템 의존성
brew install portaudio ffmpeg
```

Linux 사용자

Ubuntu/Debian에서는 `sudo apt install portaudio19-dev ffmpeg` 을 실행합니다.

`portaudio` 는 마이크 입력과 오디오 재생을, `ffmpeg` 는 오디오 포맷 변환을 담당합니다. API 키 없이 로컬에서 모든 것이 작동합니다.

빠르게 시작하기

```
$ hermes
> /voice on
```

`/voice on` 을 입력하면 음성 모드가 활성화됩니다. 이제 **Ctrl+B**를 누르고 말하면 됩니다.

1. **Ctrl+B** — 비프음(880Hz)이 울리고 녹음 시작
2. **말하기** — 화면에 오디오 레벨이 실시간 표시: `● [▁▂▃▅▇▇▅▂] ❯`
3. **말 멈추기** — 3초간 무음이 감지되면 자동으로 녹음 종료
4. **변환** — 녹음이 텍스트로 변환되어 에이전트에 전달
5. **응답** — 에이전트가 텍스트와 음성으로 동시에 답변
6. **반복** — 녹음이 자동으로 다시 시작되어, 키를 다시 누를 필요 없이 연속 대화 가능

Ctrl+B를 다시 누르면 연속 대화 모드에서 빠져나옵니다.

```
> /voice status

Voice Mode Status
  Mode:       ON
  TTS:        ON
  Recording:  no
  Record key: Ctrl+B
```

STT: 음성을 텍스트로

STT(Speech-to-Text)는 마이크 입력을 텍스트로 변환합니다. 기본값은 `local` 이며, faster-whisper를 사용해서 API 키 없이 로컬에서 작동합니다.

프로바이더	모델	비용	특징
local (기본)	base (~150MB)	무료	faster-whisper, 첫 실행 시 모델 자동 다운로드
local_command	—	무료	`HERMES_LOCAL_STT_COMMAND` 환경 변수로 커스텀 로컬 명령 지정
groq	whisper-large-v3-turbo	무료 티어	클라우드, 매우 빠름
openai	whisper-1	유료	클라우드, 안정적
xai	grok-stt	유료	xAI Grok STT, 고정밀 + 화자 분리(diarization)

> 💡 ***mistral*** *(Voxtral Transcribe) 프로바이더도 사용할 수 있습니다. 한동안* `mistralai` *PyPI 패키지가 공급망 보안 이슈로 격리됐었지만, 검토를 마친* `mistralai==2.4.8` *이 다시 설치됩니다.*

프로바이더를 지정하지 않으면 **local → local_command → groq → openai → xai → elevenlabs** 순서로 자동 폴백합니다 (mistral은 자동 탐지에서 건너뜁니다. `provider: mistral` 로 명시하면 사용할 수 있습니다).

로컬 모델 크기 선택

모델	크기	용도
`tiny`	~75MB	빠른 테스트용
`base`	~150MB	일상 사용 (기본값)
`small`	~500MB	더 정확한 인식
`medium`	~1.5GB	다국어, 고품질
`large-v3`	~3GB	최고 품질

TTS: 텍스트를 음성으로

TTS(Text-to-Speech)는 에이전트의 응답을 음성으로 변환합니다. 기본값은 Edge TTS이며, 무료이고 API 키가 필요 없습니다.

프로바이더	비용	특징
edge (기본)	무료	322개 음성, 74개 언어
piper	무료	로컬 실행, 빠르고 가벼운 ONNX 기반
neutts	무료	로컬 실행, `espeak-ng` 필요
kittentts	무료	경량 로컬 TTS
elevenlabs	유료	최고 음질
openai	유료	gpt-4o-mini-tts
xai	유료	xAI 커스텀 보이스
minimax	유료	MiniMax speech-02-hd
gemini	유료	Google Gemini TTS

> 💡 ***mistral*** *(Voxtral TTS) 프로바이더도 STT와 마찬가지로* `mistralai==2.4.8` *로 사용할 수 있습니다.*

Edge TTS 한국어 음성을 사용하려면:

```
tts:
  provider: "edge"
  edge:
    voice: "ko-KR-SunHiNeural"    # 여성, 한국어
    # voice: "ko-KR-InJoonNeural" # 남성, 한국어
```

Edge TTS 음성 목록 확인

`edge-tts --list-voices | grep ko-KR`로 사용 가능한 한국어 음성을 확인할 수 있습니다. 영어 음성은 `en-US`로 필터링합니다.

에이전트가 응답을 생성하면서 **문장 단위로 즉시 음성을 재생**합니다. 전체 응답이 끝날 때까지 기다릴 필요가 없습니다.

플러그형 TTS 레지스트리

v0.12부터 TTS 프로바이더가 `tts.providers.<name>` 레지스트리로 정리됐습니다. 새 TTS 백엔드를 플러그인으로 추가하기가 쉬워졌고, 사용자도 같은 패턴으로 직접 등록할 수 있습니다. Piper 같은 로컬 백엔드도 이 레지스트리를 통해 들어옵니다.

TUI 음성 모드

새 Ink-based TUI에서도 클래식 CLI와 동일한 음성 경험을 제공합니다. VAD(음성 활동 감지) 루프와 TTS 출력이 그대로 작동하고, 음성 관련 충돌이 났을 때를 대비한 크래시 포렌식 로그도 포함되어 있어 문제가 생기면 `~/.hermes/logs/`에서 확인할 수 있습니다. 사용 절차는 클래식 REPL과 동일합니다.

설정

`~/.hermes/config.yaml`에서 음성 관련 설정을 조정할 수 있습니다.

```
voice:
  record_key: "ctrl+b"          # 녹음 시작/종료 키
  max_recording_seconds: 120     # 최대 녹음 시간(초)
  silence_threshold: 200         # 무음 판단 기준 (높을수록 둔감)
  silence_duration: 3.0          # 무음 후 자동 종료까지 대기 시간(초)

stt:
  provider: "local"
  local:
    model: "base"

tts:
  provider: "edge"
  edge:
    voice: "ko-KR-SunHiNeural"
```

무음 감지 조정

에이전트가 말이 끝나기 전에 끊는다면 `silence_duration`을 늘립니다. 반대로 반응이 느리다면 줄입니다.

```
voice:
  silence_duration: 4.0     # 문장 사이 쉼이 긴 경우
```

주변 소음이 심하면 `silence_threshold`를 높여서 배경 소음을 무시하게 합니다.

```
voice:
  silence_threshold: 300    # 시끄러운 환경
```

녹음 키 변경

Ctrl+B가 터미널이나 tmux 단축키와 충돌하면 다른 키로 바꿀 수 있습니다.

```
voice:
  record_key: "ctrl+space"
```

실습: 음성으로 대화하기

음성 모드를 직접 체험해 봅니다.

1단계: 음성 모드 활성화

```
$ hermes
> /voice on
```

2단계: Ctrl+B를 누르고 말합니다.

```
(Ctrl+B 누른 뒤) "안녕, 오늘 날씨 어때?"
```

에이전트가 텍스트로 답하면서 동시에 음성으로도 들려줍니다.

3단계: 이어서 말해 봅니다. 연속 대화 모드이므로 키를 다시 누를 필요 없이 바로 말하면 됩니다.

```
"그러면 이번 주말 서울 날씨도 알려줘"
```

4단계: 대화가 끝나면 Ctrl+B를 눌러 음성 모드를 비활성화하거나, `/voice off`를 입력합니다.

> 음성이 안 나온다면
>
> `/voice status`로 TTS가 켜져 있는지 확인하세요. `/voice tts`로 TTS만 따로 토글할 수 있습니다. "No audio device found" 오류가 나오면 `portaudio`가 설치되었는지 확인합니다.

음성 모드 슬래시 명령어

명령어	설명
`/voice`	음성 모드 토글
`/voice on`	음성 모드 켜기
`/voice off`	음성 모드 끄기
`/voice tts`	TTS 출력만 토글
`/voice status`	현재 상태 확인

메신저에서의 음성

Telegram이나 Discord에서도 음성 메시지를 주고받을 수 있습니다. 메신저에서 보낸 음성 메시지를 STT로 변환하고, 응답을 TTS로 음성 버블로 돌려줍니다. Discord 음성 채널에 봇이 직접 참여하는 라이브 대화도 가능합니다. 자세한 설정은 9장에서 다룹니다. (→ 9장)